Compilations Studio

Turn @Arab's catalog into ready-to-edit compilation videos

@Arab

What this tool does

You give it a theme. Claude scans every transcript, returns the best moments with timestamps anchored to sentence boundaries, and lets you assemble a clip list. Export a CSV your editor can work from directly.

106 longform @Arab videos

4,305 transcript chunks indexed

53 hours of source footage

Supabase-backed compilations + titles persist

The four tabs (in order, left to right)

01 Library — your home page

Browse all 106 longform videos. Use **AI search** (Sparkles icon) to rank by relevance to any query — try "eating crazy food" and watch rat poison and hallucination honey rise to the top with a 0–100 score badge. **+ Add video** button to ingest new @Arab uploads.

02 Workspace

Where compilations get built. Type a theme, click **Find moments**, and Claude returns clip cards grouped by video. Click a thumbnail to preview in the persistent right-side player. Click **+ Add** to assemble.

03 Titles

Brainstorm viral compilation titles. Claude proposes 8 pitches with alt titles + source video suggestions. **Save** the ones you like, or **Build** to drop straight into Workspace.

04 Saved

All in-progress compilations. Click any clip thumbnail to preview inline. **Resume** reopens it in Workspace, **Export CSV** hands it to your editor, **Delete** wipes it.

End-to-end workflow

1. Titles Brainstorm pitch → **2. Save** Bookmark → **3. Build** → Workspace → **4. Find moments** Pull clips → **5. + Add** Assemble → **6. Export CSV** To editor

Context vs Moment clips — important

Every clip is labeled as one of two kinds:

CONTEXT

20–60 second clips that set up the scenario — where Arab is, who he's with, what's at stake. Plays FIRST in the final compilation so the moment lands.

MOMENT

30–180 second clips that ARE the payoff — the shocking line, the action, the wild reaction. The screenshot-worthy stuff. Plays after the context.

Per video, Claude returns 1–2 context clips + 2–3 moments, ordered chronologically. The CSV export includes a `kind` column so your editor knows the role.

> **Tip — Title style.** The AI avoids "I Spent X Days With Y" (those are Arab's own vlog hooks) and listicle phrasings. It produces multi-story-implying titles like "Times Brazil Got Out of Control" or "Why You Don't Mess With Favelas" — viral without screaming compilation.

Power moves, shortcuts, and tech notes

Page 2 — saves you time once you're comfortable

A

How clip extraction actually works

Each transcript is split into ~45-second chunks, but each chunk keeps the underlying YouTube auto-caption line-level timing. When you search:

- **Stage 1** — ranks all 106 video titles + descriptions for relevance
- **Stage 2** — scans the top 8 picked videos in parallel, asking for context + moment clips with start times that match a specific line boundary
- Result: clips never start mid-word, and they're labeled by role

One persistent inline player

The whole app has a single YouTube iframe. Clicking different clip thumbnails seeks via postMessage instead of reloading. Switch between 8 clips in 8 seconds without YouTube re-loading once.

Open → YouTube in new tab

The "YouTube" link on each clip card opens the full video on youtube.com in a new tab. Your workspace state stays intact. (The deep-dive video page is only reachable from clicking a card in the Library tab.)

State persists across tabs

Theme, generated ideas, search results, and active compilation are saved to localStorage. Navigate to Library, come back, everything's still there. Same after refresh.

AI Library search

Plain text search only matches title/description substrings. **AI search** sends your query + the whole 106-video index to Claude with a relevance scoring rubric.

- 90–100 dominant subject
- 70–89 strong match
- 40–69 touched on
- under 25 filtered out

Adding new @Arab uploads

Library → **+ Add video**. Paste any YouTube URL or 11-character ID. The server:

1. Fetches metadata via YouTube Data API
2. Validates channel = @Arab (rejects everything else)
3. Shells out to yt-dlp for auto-captions
4. Parses VTT, dedupes rolling captions, chunks at ~45s
5. Appends to catalog and chunks files

Available in AI search instantly.

Supabase backend

Compilations and titles persist in your Vision CRM Supabase project, in an isolated set of tables (arab_compilations, arab_clips, arab_titles). RLS is on. Catalog + transcripts remain static JSON in the repo since they don't change often.

CSV export columns

#, kind (context or moment), video_title, video_url, youtube_url_with_start, in_tc, out_tc, duration_sec, note (the quoted line). Drag into Premiere/Resolve/CapCut as a paper edit.

Titles vs Saved

- **Titles** = pitches you saved but haven't built — bookmarks
- **Saved** = actual compilations with picked clips ready to export

Keyboard shortcuts

- `Enter` in workspace input — Find moments
- `Enter` in titles input — Generate ideas
- `Esc` in any modal — Cancel
- `Enter` in delete modal — Confirm

Mobile

Fully responsive. On phones: workspace + titles inputs stack, saved cards put actions in a row below the content, the right-side player drops below the main column. All modals fit a 360px viewport.

Visual cheat sheet

Workspace 02

Type theme → **Find moments**

› Click thumbnail → plays in right panel

› **+ Add** → into compilation

Titles 03

Seed theme → **Generate ideas**

› **Save** bookmarks, **Build** opens workspace

› Click thumb in idea → inline player

Saved 04

Each comp shows clip thumb strip

› Click any thumb → inline preview

› **Resume** / **Export CSV** / **Delete**

Tip — Compilation pacing. Target 15–60 minutes total. A clean pattern is context → moment → context → moment for 6–10 videos. The AI returns clips chronologically so the natural assembly already works. Click each thumbnail to preview before

adding so you know it delivers.